Session 15 Homework — The Real Data Foundation Session 16 Will Build On

CORE, STRETCH, and CHALLENGE are all required — every one of them must be completed. The three names describe increasing difficulty only, nothing about which parts you submit.

Context

By the end of class, every write route across Issues, Projects, and Workspaces requires a real signed-in user — that question (WHO is making this request) is fully answered. This homework does **not** protect another resource; every resource is already protected. Instead, it begins the real next question this app will need to answer: **once we know WHO someone is, how do we know WHAT they belong to, and in what role?** You will build the real, minimal data model this app's own frozen design calls `WorkspaceMember` — the join between a `User` and a `Workspace` that will carry role. **You are building data only. No endpoint, no guard, no permission check, no enforcement of any kind.** Nothing in this homework decides who is allowed to do what — it only makes the FACTS that decision will eventually need actually exist and actually be correct.

Goal

A real `WorkspaceMember` record persists in the database every time a workspace is created through the real, authenticated API, correctly identifying the workspace, the real signed-in user who created it, and that user's role as the workspace's Owner. A real lookup answers "what is this user's role in this workspace" and is proven to be actually used by the database, not just present in code. A second real lookup answers the reverse question. A real, enforced rule guarantees no user can ever end up with two different roles in the same workspace. And you can explain, in writing, why this shape — and not a simpler one — is the right one.

What to build

CORE — the `WorkspaceMember` entity itself, wired into the one real place it should be created today, proven against a real, running database. **STRETCH** — the reverse lookup direction, proven the same way. **CHALLENGE** — written reasoning connecting this homework's own real design choices to the authorization system Session 16 will actually build on top of them.

CORE (required)

Build the real `WorkspaceMember` data foundation:

- A new, real, database-backed entity representing "this specific user's membership in this specific workspace," following this app's own already-frozen domain design exactly — the entity's name, its field names, and its role values are not yours to invent; find them in this project's own domain documentation and match them precisely, including capitalization.

- A real database constraint guaranteeing that the same user can never have two separate membership rows in the same workspace — this must be enforced at the database layer, not only checked in application code, because application-level checks alone cannot prevent two near-simultaneous requests from both succeeding.
- A real, callable lookup capability that, given a specific workspace and a specific user, returns that user's own membership in that workspace (or confirms none exists) — this is the exact question Session 16 will need answered on every protected action, so it must be a real, reusable method, not inline logic duplicated wherever it's needed.
- A real trigger: wire this new capability into the one place in this app where a membership genuinely needs to be created today — when a workspace is created through the real, authenticated API, its own creator must end up with a real membership row identifying them by their own role in this app's three-role model, specifically the role meaning "created this workspace, has full control of it." Use the identity already available from the request itself (the same mechanism already built live this session for reading who's making a request) — never a hard-coded or invented user id.
- **No new HTTP endpoint.** Nothing about this homework requires exposing membership data or membership creation directly over the API yet — if you find yourself adding a route, stop; you've gone further than CORE requires.
- Prove it against a real, running database: a real workspace created through a real authenticated request must leave behind a real, inspectable record with the correct workspace, the correct user, and the correct role. Prove the lookup capability is actually served by the database constraint you built, not merely "probably using it." Prove the uniqueness rule by attempting a genuine duplicate directly against the database and confirming it is rejected.

STRETCH (required)

Add the reverse lookup direction — given a specific user, find every workspace they're a member of — and prove, the same rigorous way as CORE, that it is backed by its own real database support rather than silently reusing (and therefore secretly relying on, inefficiently, the wrong way around) whatever CORE already built for the forward direction. Prove it against a user who is a member of more than one real workspace, not just one.

CHALLENGE (required)

Write short, direct, written answers, placed directly above the new entity you built in CORE:

1. Your `JwtAuthGuard` (already built, already proven) answers one question. The entity you just built exists to eventually help answer a different one. Name both questions precisely, and state which one this homework actually implements an answer to — not just "prepares for."
2. Why does role belong on the new entity you built, rather than directly on `User`? Make your answer concrete: describe a real situation, using this app's own actual domain, where putting role directly on `User` would be unable to represent something that's actually true.
3. This app's own frozen design stores membership as its own separate, real, queryable entity, rather than, say, a list embedded inside `Workspace`. Give one real, concrete reason this matters for a question this app will genuinely need to answer later.

4. You built a real lookup capability in CORE. Precisely what is still missing — name it concretely, in terms of this app's own real routes and real guards — before anything in this app actually stops an unauthorized action from happening? Your answer should make clear you understand that what you built this week is necessary but not sufficient.

Rules

Allowed: Mongoose schema/service/module patterns (already used throughout this course), `@CurrentUser()` (already built and proven live this session — you may use it somewhere other than where it was first built), everything already shown in every prior session. No new packages.

Not yet: no HTTP endpoint for the new entity, no guard or decorator that reads a role or membership to allow or deny a request, no decorator-based role-check mechanism of any kind, no role-based or ownership-based enforcement anywhere, no changes to any existing route's own guard.

Test it

Never assume a user's own id or a workspace's own id is any particular number. A freshly registered user is NOT guaranteed to get id `1` — seeded data, or any earlier activity on the same database (your own teammates testing, an earlier run of your own proof), already shifts it, and the same is true for a freshly created workspace. Capture the real ids from the real responses every time, and use those captured values — never a literal number — in every proof below.

BASH

```
# register, log in, capture YOUR OWN real user id, create a workspace, capture ITS real id
curl -s -X POST http://localhost:3000/auth/register -H "Content-Type: application/json" \
  -d '{"name":"Check","email":"hw-check@example.com","password":"password123"}'
TOKEN=$(curl -s -X POST http://localhost:3000/auth/login -H "Content-Type: application/json"
\
  -d '{"email":"hw-check@example.com","password":"password123"}' | python3 -c "import
json,sys;print(json.load(sys.stdin)['accessToken'])")
USER_ID=$(curl -s http://localhost:3000/auth/me -H "Authorization: Bearer $TOKEN" \
  | python3 -c "import json,sys;print(json.load(sys.stdin)['id'])")
WORKSPACE_ID=$(curl -s -X POST http://localhost:3000/workspaces -H "Content-Type:
application/json" \
  -H "Authorization: Bearer $TOKEN" -d '{"name":"Check WS","slug":"hw-check-ws"}' \
  | python3 -c "import json,sys;print(json.load(sys.stdin)['id'])")
echo "user id: $USER_ID, workspace id: $WORKSPACE_ID"

# inspect the real persisted membership record directly, by its REAL ids — confirm it
exists, and is correct
mongosh b4f-board --eval "db.workspacemembers.findOne({workspaceId: $WORKSPACE_ID, userId:
$USER_ID})"

# confirm the forward lookup is real and index-backed (not a collection scan)
mongosh b4f-board --eval "db.workspacemembers.find({workspaceId: $WORKSPACE_ID, userId:
$USER_ID}).explain('executionStats')"

# attempt a genuine duplicate of that SAME real pair directly against the database — this
must fail
mongosh b4f-board --eval "db.workspacemembers.insertOne({id:9999, workspaceId:$WORKSPACE_ID,
userId:$USER_ID, role:'member', joinedAt:new Date()})"
```

Design your own equivalent proof for the STRETCH reverse direction, using a user who created more than one real workspace, and that same user's own real captured id — not an assumed number.

Done when

- Creating a workspace through the real, authenticated API leaves behind a real, correct membership record — proven by inspecting the actual database, not just reading the HTTP response.
- Every proof uses the REAL captured user id and workspace id from that exact run — never a hard-coded number. Your own user's id is unlikely to be `1`; the seeded Workspaces already use ids `1` and `2`, so a correct proof naturally lands elsewhere.
- The forward lookup capability is proven correct AND proven to actually use a real database index (not a full collection scan).
- A genuine duplicate membership attempt, made directly against the database, is actually rejected.
- The reverse lookup capability (STRETCH) is proven correct and index-backed for a user with more than one real membership.

- No HTTP endpoint, guard, decorator, or any form of permission check reads or enforces a role anywhere in the app — this is verifiable by inspecting your own diff, not just by your own claim.
- Your CHALLENGE answers are grounded in this app's own real fields and real gaps, not a generic explanation of role-based access control.

Working time

About 90–120 minutes — this is intentionally substantial. CORE is a real new domain capability (schema, service, a real trigger, real database proof), not three decorators; STRETCH is a second real query direction; CHALLENGE is short but requires actually understanding the gap between what exists now and what Session 16 will still need to build.